PES-VCS Report

Version Control System from Scratch

NAME : Priyanka B M

SRN: PES1UG24AM207

Course: Operating Systems

Assignment: PES-VCS Implementation

Phase 1 – Object Storage Foundation

1A : `./test_objects` output showing all tests passing

```
pes1ug24am207@Ubuntu:~/PES1UG24AM207-pes-vcs$ make test_objects
gcc -Wall -Wextra -O2 -c object.c -o object.o
gcc -o test_objects test_objects.o object.o -lcrypto
pes1ug24am207@Ubuntu:~/PES1UG24AM207-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
pes1ug24am207@Ubuntu:~/PES1UG24AM207-pes-vcs$
```

1B : `find .pes/objects -type f` showing sharded directory structure

```
pes1ug24am207@Ubuntu:~/PES1UG24AM207-pes-vcs$ find .pes/objects -type f
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
pes1ug24am207@Ubuntu:~/PES1UG24AM207-pes-vcs$
```

Phase 2 – Tree Objects

2A : `/test_tree` output showing all tests passing

```
pes1ug24am207@Ubuntu:~/PES1UG24AM207-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
pes1ug24am207@Ubuntu:~/PES1UG24AM207-pes-vcs$
```

2B : xxd of a raw tree object (first 20 lines)

```
pes1ug24am207@Ubuntu:~/PES1UG24AM207-pes-vcs$ find .pes/objects -type f
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
pes1ug24am207@Ubuntu:~/PES1UG24AM207-pes-vcs$ xxd .pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
00000000: 626c 6f62 2031 3800 4475 706c 6963 6174  blob 18.Duplicat
00000010: 6520 636f 6e74 656e 740a                 e content.
pes1ug24am207@Ubuntu:~/PES1UG24AM207-pes-vcs$
```

Phase 3 – Index (Staging Area)

3A : `pes init` → `pes add` → `pes status` sequence

```
pes1ug24am207@Ubuntu:~/PES1UG24AM207-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
pes1ug24am207@Ubuntu:~/PES1UG24AM207-pes-vcs$ echo "hello" > file1.txt
pes1ug24am207@Ubuntu:~/PES1UG24AM207-pes-vcs$ echo "world" > file2.txt
pes1ug24am207@Ubuntu:~/PES1UG24AM207-pes-vcs$ ./pes add file1.txt file2.txt
pes1ug24am207@Ubuntu:~/PES1UG24AM207-pes-vcs$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  index.c
  untracked:  tree.h
  untracked:  commit.h
  untracked:  test_objects.c
  untracked:  commit.c
  untracked:  object.c
  untracked:  README.md
  untracked:  test_tree
  untracked:  index.h
  untracked:  pes.c
  untracked:  test_tree.c
  untracked:  test_sequence.sh
  untracked:  tree.c
  untracked:  Makefile
  untracked:  pes.h
  untracked:  test_objects
  untracked:  .gitignore
```

3B : `cat .pes/index` showing the text-format index

```
pes1ug24am207@Ubuntu:~/PES1UG24AM207-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776759556 6 file1.txt
100644 e00c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f 1776759573 6 file2.txt
```

Phase 4 – Commits and History

4A : `pes log` output with three commits

```
pes1ug24am207@Ubuntu:~/PES1UG24AM207-pes-vcs$ ./pes log
commit bc85fa815742564f353795d537db1255ddf364737cec21137b96561288500149
Author: PES User <pes@localhost>
Date:   1776760963

    Add farewell

commit 18613f3b008ca5eb729ff5d6b7ef9b49677c38534b633d3a95c08fbf4cfa5008
Author: PES User <pes@localhost>
Date:   1776760915

    Add worldell

commit d58945a30a3f89e1a1cc9a70e0929966ec982d24edc272e278e76b475c00cc5b
Author: PES User <pes@localhost>
Date:   1776760862

    Initial committial commiQ
```

4B : showing object growth

```
pes1ug24am207@Ubuntu:~/PES1UG24AM207-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/18/613f3b008ca5eb729ff5d6b7ef9b49677c38534b633d3a95c08fbf4cfa5008
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/81/065d8ae4ebf80941f86e4cf1ccf0500ab07f55180cd1f2a3f44af5b4257818
.pes/objects/85/e367b76c2417b47a95f46208835ad8700dd58335171c94a497974be6191900
.pes/objects/bc/85fa815742564f353795d537db1255ddf364737cec21137b96561288500149
.pes/objects/be/03a393adb03a84878402ea8937c2af2b745821c23f413865d2f36fbf2e2515
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/d5/8945a30a3f89e1a1cc9a70e0929966ec982d24edc272e278e76b475c00cc5b
.pes/objects/e0/0c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f
.pes/refs/heads/main
```

4C `cat .pes/refs/heads/main` and `cat .pes/HEAD`

```
pes1ug24am207@Ubuntu:~/PES1UG24AM207-pes-vcs$ cat .pes/refs/heads/main
bc85fa815742564f353795d537db1255ddf364737cec21137b96561288500149
pes1ug24am207@Ubuntu:~/PES1UG24AM207-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
```

Analysis Questions

Branching and Checkout

Q5.1: A branch in Git is just a file in `.git/refs/heads/` containing a commit hash. Creating a branch is creating a file. Given this, how would you implement `pes checkout <branch>` — what files need to change in `.pes/`, and what must happen to the working directory? What makes this operation complex?

ANS: To implement `pes checkout <branch>`, the system must first check whether the branch exists inside `.pes/refs/heads/`. Each branch file stores the latest commit hash of that branch. When checkout is performed, the `.pes/HEAD` file must be updated so that it points to the selected branch reference, for example:

```
ref: refs/heads/<branch>
```

After changing `HEAD`, the commit hash stored in the target branch file is read. That commit points to a tree object representing the project snapshot. The working directory must then be updated to match that tree. Files present in the target branch should be created or overwritten, files deleted in that branch should be removed, and directories should be recreated as required. The index file should also be refreshed to match the checked-out commit.

This operation becomes complex because the system must safely replace the current working directory contents while preserving user data. If the user has modified files that are not committed, switching branches may overwrite those changes. It must also correctly handle nested folders, file deletions, permissions, and conflicts between branches.

Q5.2: When switching branches, the working directory must be updated to match the target branch's tree. If the user has uncommitted changes to a tracked file, and that file differs between branches, checkout must refuse. Describe how you would detect this "dirty working directory" conflict using only the index and the object store.

ANS: To detect a dirty working directory conflict, the system must compare the current working directory files with the entries stored in the index. The index contains information such as file path, size, modified time, and hash. If any tracked file has a different size, modified time, or content hash compared to the index entry, it means the file has uncommitted local changes.

Next, the system must compare the target branch tree stored in the object store with the current branch tree. If the same file is different in the target branch and the user has also modified it locally, switching branches would overwrite those uncommitted changes.

In this case, checkout must be refused and the user should be asked to commit or stash changes first. This prevents accidental data loss while changing branches.

Q5.3: "Detached HEAD" means HEAD contains a commit hash directly instead of a branch reference. What happens if you make commits in this state? How could a user recover those commits?

ANS: Detached HEAD means the `HEAD` file points directly to a commit hash instead of a branch name. If commits are made in this state, new commits are created normally, but they are not attached to any branch. These commits can become unreachable later if the user switches branches, because no branch reference points to them.

To recover those commits, the user can create a new branch pointing to the detached commit hash. This saves the commit history and makes the commits reachable again. If the hash is forgotten, it can often be found using the reflog or commit history records.

Garbage Collection and Space Reclamation

Q6.1: Over time, the object store accumulates unreachable objects — blobs, trees, or commits that no branch points to (directly or transitively). Describe an algorithm to find and delete these objects. What data structure would you use to track "reachable" hashes efficiently? For a repository with 100,000 commits and 50 branches, estimate how many objects you'd need to visit.

ANS: To find unreachable objects, start from all branch references in `.pes/refs/heads/` and read the commit hashes. From each commit, recursively visit its parent commits, tree objects, and blob objects. Mark every visited object as reachable. After traversal is complete, scan the object store and delete any object that is not marked reachable.

A hash set is the best data structure to store reachable hashes because lookup is fast and duplicates are avoided.

For a repository with 100,000 commits and 50 branches, around 100,000 commit objects must be visited, along with their related tree and blob objects. The total number of visited objects could be several hundred thousand depending on repository size

Q6.2: Why is it dangerous to run garbage collection concurrently with a commit operation? Describe a race condition where GC could delete an object that a concurrent commit is about to reference. How does Git's real GC avoid this?

ANS: Running garbage collection during a commit is dangerous because GC may delete objects that are being created but are not yet referenced by any branch. For example, a commit process may write new blob and tree objects first, then create the commit object, and

finally update the branch reference. If GC runs before the branch is updated, it may treat those new objects as unreachable and delete them.

This creates a race condition where the commit later points to missing objects, causing repository corruption.

Git avoids this by using locks, temporary files, atomic reference updates, and safe garbage collection rules. It usually does not delete recently created loose objects immediately, which prevents removing objects that are part of an ongoing commit operation.